Machine Intelligence 1 Tutorial

08 Elements of Statistical Learning Theory

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 25/26

Linear separability

Convergence of Empirical Risk Minimization (ERM)

The Vapnik-Chervonenkis (VC) dimension

The growth function

Results of statistical learning theory

References

Linear separability

Setting:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_{\mathrm{T}}^{(\alpha)} \right) \right\}, \quad \alpha = 1, \ldots, p$$

- p points
- N dimensions $\underline{\mathbf{x}} \in \mathbb{R}^N$
- binary assignment: $y_T \in \{-1, 1\}$

Linear separability

Setting:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_{\mathrm{T}}^{(\alpha)} \right) \right\}, \quad \alpha = 1, \ldots, p$$

- p points
- N dimensions $\underline{\mathbf{x}} \in \mathbb{R}^N$
- binary assignment: $y_T \in \{-1, 1\}$

Model:

Linear neuron with N weights and a bias ($N + 1$ degrees of freedom).

Linear separability

Setting:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_{\mathrm{T}}^{(\alpha)} \right) \right\}, \quad \alpha = 1, \ldots, p$$

- p points
- N dimensions $\underline{\mathbf{x}} \in \mathbb{R}^N$
- binary assignment: $y_T \in \{-1, 1\}$

Model:
Linear neuron with N weights and a bias ($N + 1$ degrees of freedom).
Task:
Find the total number of binary assignments that are linearly separable.

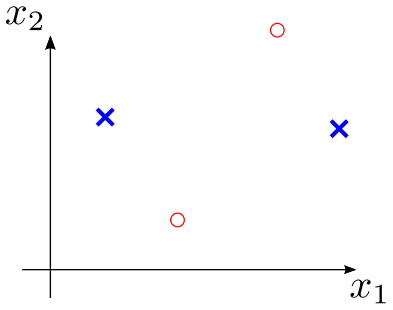
Find the total number of affinely separable assignments

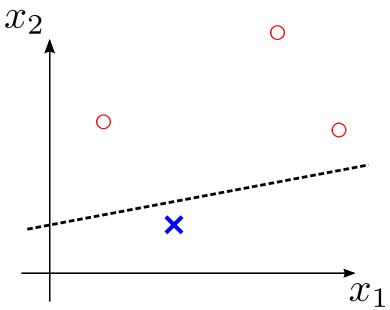
1. Place p points in N dimensions, any way you like as long as they're not collinear.
2. Assign all possible labels to the points.

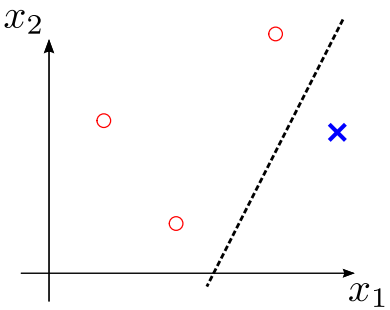
Example with 4 points in 2D:
see blackboard...

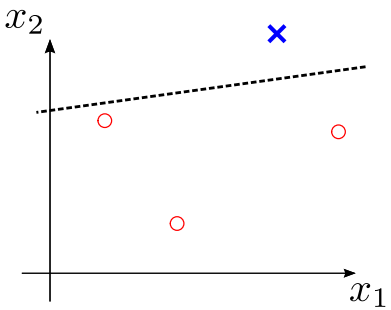
Q: *How many assignment configurations are there?*

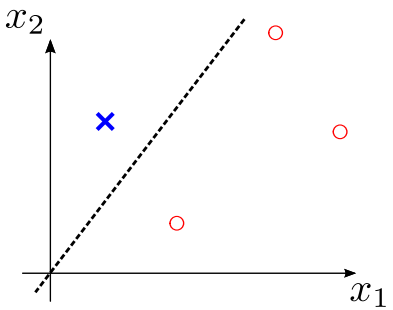
Q: *How many of them can be separated by a connectionist neuron?*

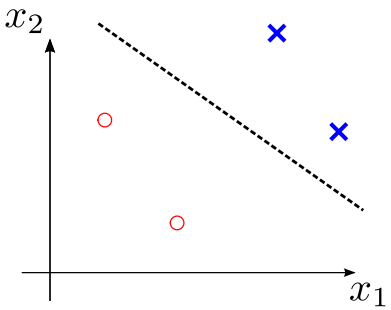

x_2
x_1

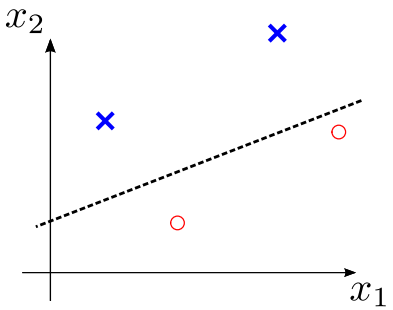

x_2
x_1

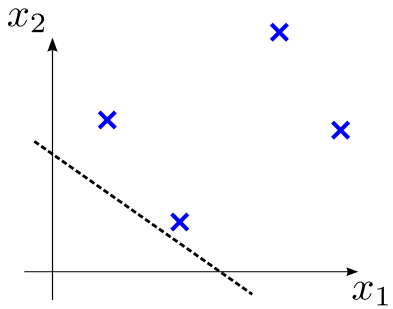

x_2
x_1


x_2
x_1


x_2
x_1


x_2
x_1


x_2
x_1


x_2
x_1

Calculate the number of configurations that are linearly separable

Calculate the number of linearly separable assignments:

$$\tilde{C}_{(p,N)} := 2 \sum_{k=0}^{N} \underbrace{\binom{p-1}{k}}_{\substack{\text{Binomial} \\ \text{coefficient}}}$$

The Binomial coefficient[1] ("n choose k") is defined as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

for $n, k \in \mathbb{N}_0$ with $n \geq k \geq 0$

[1] See Pascal's triangle for a fun way on how to manually compute the coefficient.

Calculating the number of separable configurations?

$$\tilde{C}_{(p,N)} := 2 \sum_{k=0}^{N} \binom{p-1}{k}$$

Using our example with $p = 4$ and $N = 2$:

$$\begin{aligned}
\tilde{C}_{(4,2)} &= 2 \sum_{k=0}^{2} \binom{4-1}{k} \\
&= 2 \left[\binom{4-1}{0} + \binom{4-1}{1} + \binom{4-1}{2} \right] \\
&= 2 \left[1 + 3 + 3 \right] \\
&= 14
\end{aligned}$$

$$\tilde{C}_{(p,N)} := 2 \sum_{k=0}^{N} \binom{p-1}{k}$$

But this looks slightly different from what is in the lecture. In the lecture it looked like this:

$$C_{(p,N)} := 2 \sum_{k=0}^{N-1} \binom{p-1}{k}$$

Q: *Why does the sum for $\tilde{C}_{(p,N)}$ run to N instead of $N-1$?*

Recursive formula for the binomial coefficient:

$$\begin{pmatrix} n \\ k \end{pmatrix} + \begin{pmatrix} n \\ k-1 \end{pmatrix} = \begin{pmatrix} n+1 \\ k \end{pmatrix}$$

Use the the recursive formula for the binomial coefficient to show that:

$$\tilde{C}_{(p,N)} + \tilde{C}_{(p,N-1)} = \tilde{C}_{(p+1,N)}$$

The relevance of this relationship becomes apparent when attempting to derive the VC dimension of a connectionist neuron.

Show that

$$\tilde{C}_{(p,N)} \quad + \quad \tilde{C}_{(p,N-1)} \quad = \quad \tilde{C}_{(p+1,N)}$$

$$2\sum_{k=0}^{N}\binom{p-1}{k} + 2\sum_{k=0}^{N-1}\binom{p-1}{k} = 2\sum_{k=0}^{N}\binom{p+1-1}{k}$$

Show that

$$\tilde{C}_{(p,N)} \quad + \quad \tilde{C}_{(p,N-1)} \quad = \quad \tilde{C}_{(p+1,N)}$$

$$2\sum_{k=0}^{N}\binom{p-1}{k} + 2\sum_{k=0}^{N-1}\binom{p-1}{k} = 2\sum_{k=0}^{N}\binom{p+1-1}{k}$$

Simplify the LHS by modifying the sums s.t. they share the same limits.

$$2\binom{p-1}{0} + 2\sum_{k=\cancel{0}1}^{N}\binom{p-1}{k} + 2\sum_{k=0}^{N-1}\binom{p-1}{k} = 2\sum_{k=0}^{N}\binom{p}{k}$$

$$2\sum_{k=0}^{N}\binom{p-1}{k}+2\sum_{k=0}^{N-1}\binom{p-1}{k}=2\sum_{k=0}^{N}\binom{p+1-1}{k}$$

Simplify the LHS by modifying the sums s.t. they share the same limits.

$$2\binom{p-1}{0}+2\sum_{k=\cancel{0}1}^{N}\binom{p-1}{k}+2\sum_{k=0}^{N-1}\binom{p-1}{k}=2\sum_{k=0}^{N}\binom{p}{k}$$

$$2\binom{p-1}{0}+2\sum_{k=1}^{N}\binom{p-1}{k}+2\sum_{k=1}^{N}\binom{p-1}{k-1}=$$

$$2\sum_{k=0}^{N}\binom{p-1}{k}+2\sum_{k=0}^{N-1}\binom{p-1}{k}=2\sum_{k=0}^{N}\binom{p+1-1}{k}$$

Simplify the LHS by modifying the sums s.t. they share the same limits.

$$2\binom{p-1}{0}+2\sum_{k=\cancel{0}1}^{N}\binom{p-1}{k}+2\sum_{k=0}^{N-1}\binom{p-1}{k}=2\sum_{k=0}^{N}\binom{p}{k}$$

$$2\binom{p-1}{0}+2\sum_{k=1}^{N}\binom{p-1}{k}+2\sum_{k=1}^{N}\binom{p-1}{k-1}=$$

$$2\binom{p-1}{0}+2\sum_{k=1}^{N}\Bigg[\underbrace{\binom{p-1}{k}+\binom{p-1}{k-1}}_{=\binom{p}{k}\text{ (using the recursive property)}}\Bigg]=$$

$$2\binom{p-1}{0} + 2\sum_{k=\cancel{0}1}^{N}\binom{p-1}{k} + 2\sum_{k=0}^{N-1}\binom{p-1}{k} = 2\sum_{k=0}^{N}\binom{p}{k}$$

$$2\binom{p-1}{0} + 2\sum_{k=1}^{N}\binom{p-1}{k} + 2\sum_{k=1}^{N}\binom{p-1}{k-1} =$$

$$2\binom{p-1}{0} + 2\sum_{k=1}^{N}\Bigg[\underbrace{\binom{p-1}{k} + \binom{p-1}{k-1}}_{=\binom{p}{k}\text{ (using the recursive property)}}\Bigg] =$$

$$2\underbrace{\binom{p-1}{0}}_{=1=\binom{p}{0}} + 2\sum_{k=1}^{N}\binom{p}{k} = 2\sum_{k=0}^{N}\binom{p}{k}$$

$\binom{p}{0}$ can now be absorbed into the sum.

$$2\underbrace{\binom{p-1}{0}}_{=1=\binom{p}{0}} + 2\sum_{k=1}^{N}\binom{p}{k} = 2\sum_{k=0}^{N}\binom{p}{k}$$

$\binom{p}{0}$ can now be absorbed into the sum.

$$2\sum_{k=0}^{N}\binom{p}{k} = 2\sum_{k=0}^{N}\binom{p}{k}$$

$$\tilde{C}_{(p,N)} + \tilde{C}_{(p,N-1)} = \tilde{C}_{(p+1,N)}$$

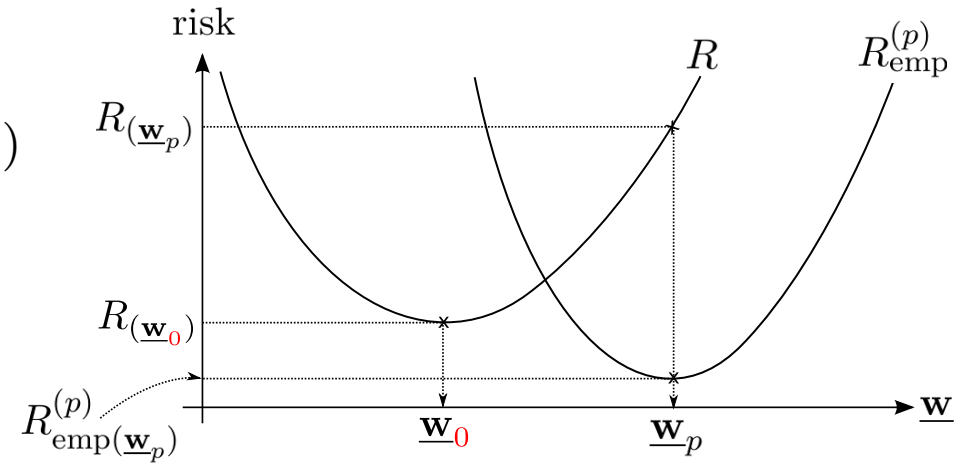
Convergence of Empirical Risk Minimization (ERM)

The learning problem

Setting:

- samples are drawn i.i.d. from $P(\underline{\mathbf{x}}, y_T)$

  $\rightsquigarrow$ $\underline{\mathbf{w}}$ are random variables
- $R_{emp} \widehat{=} E^T_{[\underline{\mathbf{w}}]}$
- $R \widehat{=} E^G_{[\underline{\mathbf{w}}]}$

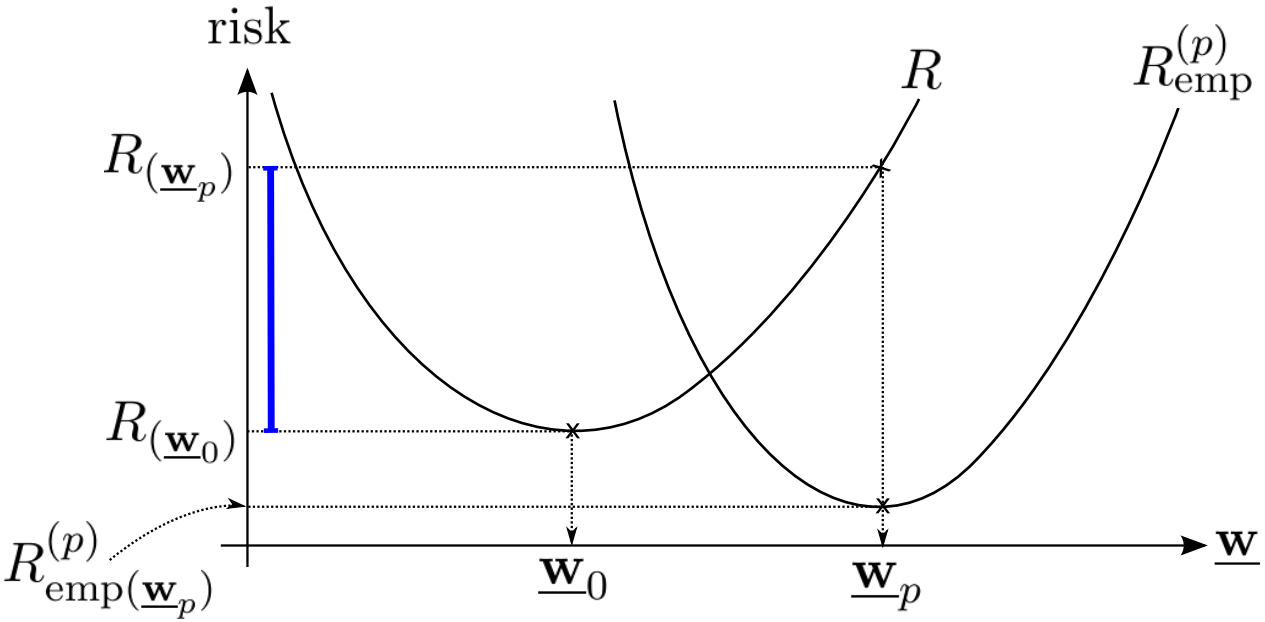


Recall that

$$
\begin{aligned}
E^G_{[\underline{\mathbf{w}}]} &= \text{probability of misclassification} \\
&= \int P_{(y_T,\underline{\mathbf{x}})}\, e_{(y_T,\underline{\mathbf{x}};\underline{\mathbf{w}})}\, d\underline{\mathbf{x}}\, dy_T \;\stackrel{!}{=}\; \min_{\underline{\mathbf{w}}}
\end{aligned}
$$

(Don't treat the $_0$ as zero data points but rather as "minimal" risk)

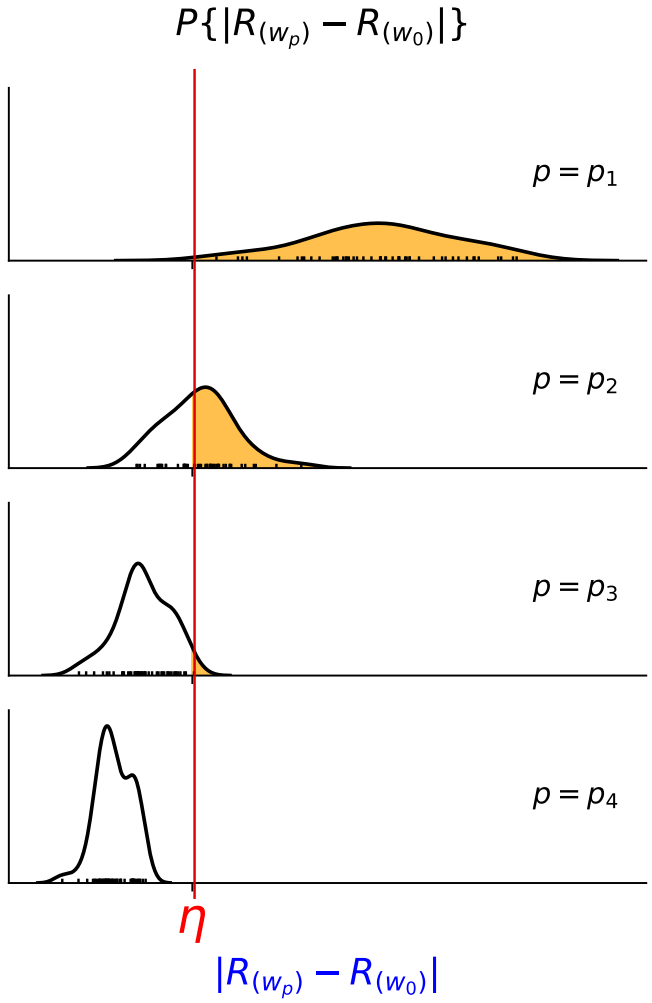
The key questions of Statistical Learning Theory



1. When does inductive learning through ERM work?
   - $R_{\text{emp}(\underline{\mathbf{w}}_p)}$ should reflect the true optimal risk $R_{(\underline{\mathbf{w}}_0)}$ in the limit $p \to \infty$:

$$\lim_{p\to\infty} P\Big\{ \big| R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)} \big| \geq \eta \Big\} \;=\; 0\,, \qquad \text{for all} \quad \eta > 0\,.$$

Understanding what $P\{|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}| \geq \eta\}$ represents



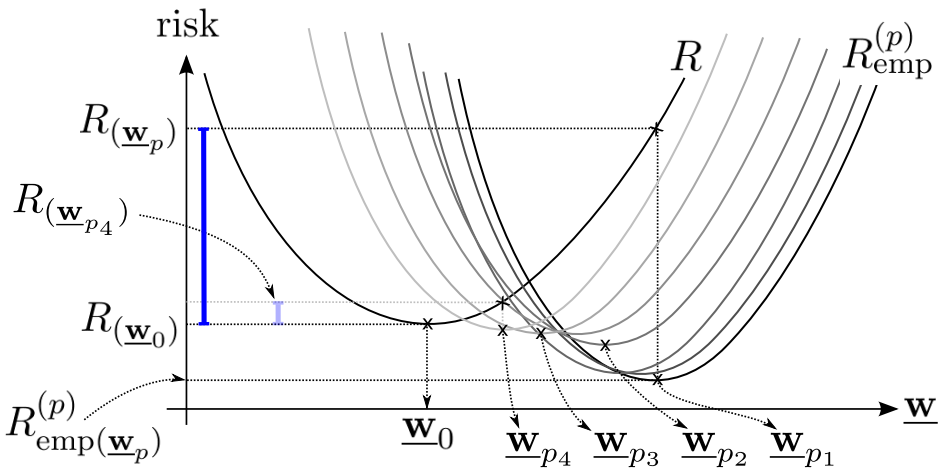
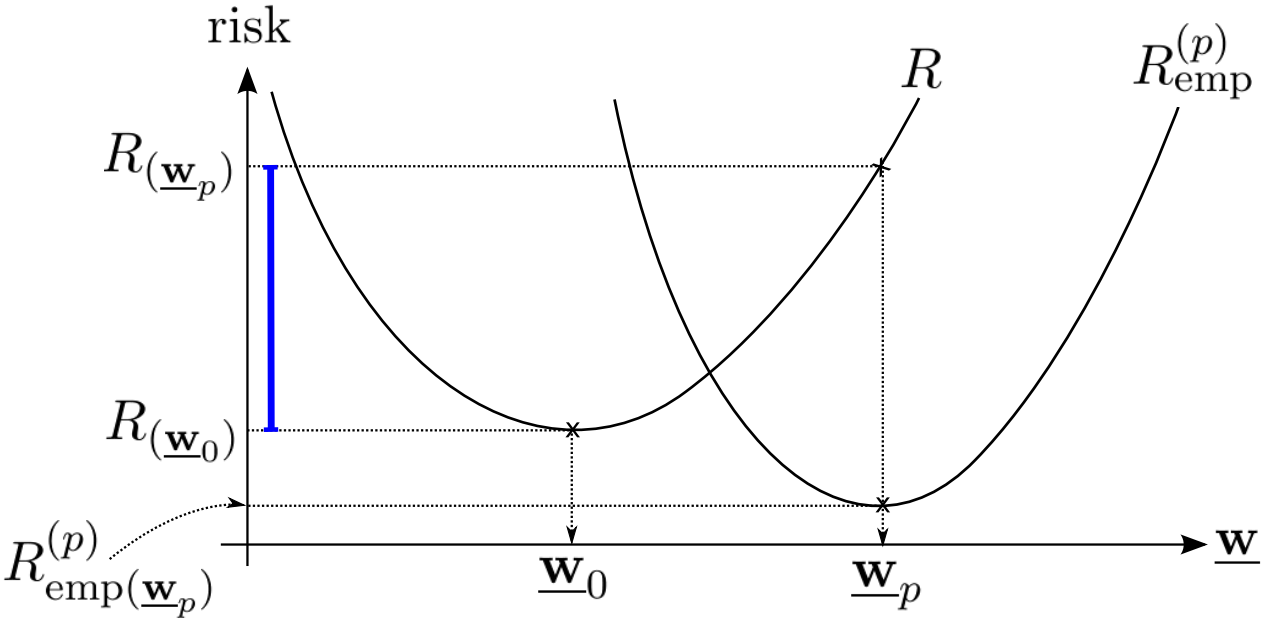


Figure: Distribution of absolute risk differences for larger number of training points $p_1 < p_2 < p_3 < p_4$

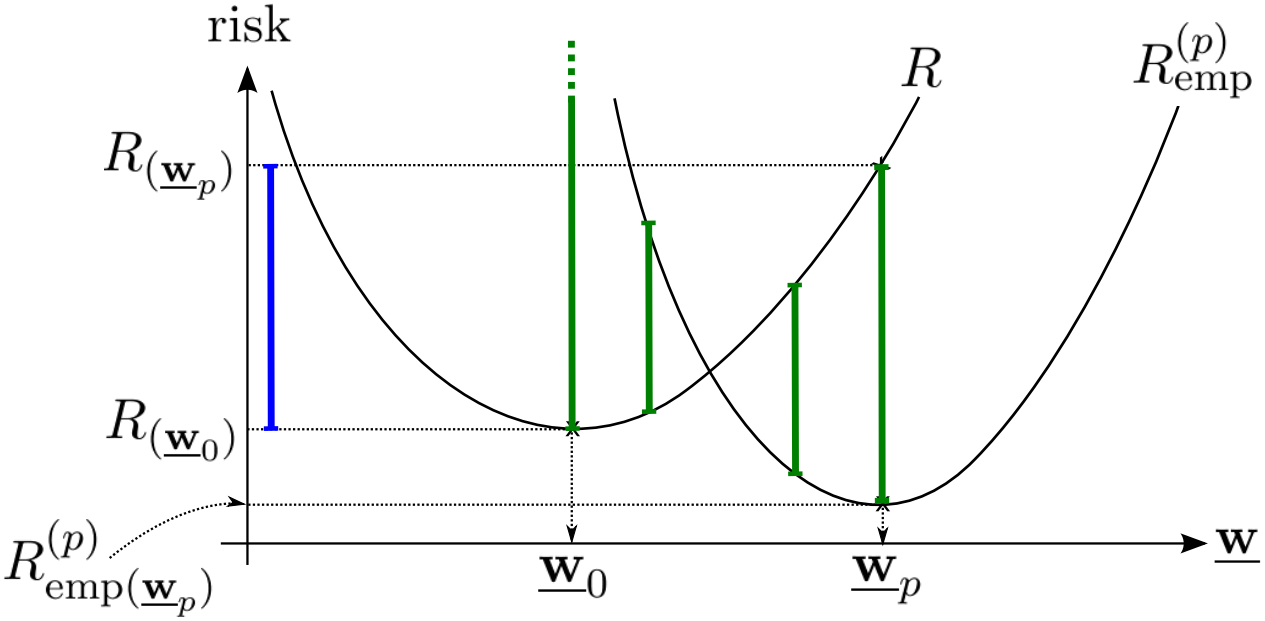
The key questions of Statistical Learning Theory



2. How strongly does $R_{(\underline{\mathbf{w}}_p)}$ differ from $R_{(\underline{\mathbf{w}}_0)}$ for **finite** samples?
   - For a given confidence ϵ, find η such that

$$P\Big\{\big|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\big| \geq \eta\Big\} \quad < \quad \epsilon\,.$$

The key questions of Statistical Learning Theory



3. Do training errors $R_{\text{emp}(\underline{\mathbf{w}})}$ reflect generalization performance $R_{(\underline{\mathbf{w}})}$?
   - Avoid overfitting for finite samples, i.e. given confidence ϵ, find η such that:

$$P\left\{\left|R_{(\underline{\mathbf{w}})} - R_{\text{emp}(\underline{\mathbf{w}})}\right| > \eta\right\} \quad < \quad \epsilon\,, \qquad \forall \underline{\mathbf{w}} \in \Lambda\,.$$

where Λ is the model class (e.g. connectionist neuron)

The Vapnik-Chervonenkis (VC) dimension

Shattering p points

Q: *Can a perceptron classify any 3 points in 2D?*

Q: *Can a perceptron classify any 4 points in 2D?*

Shattering p points

Q: *Can a perceptron classify any 3 points in 2D?*

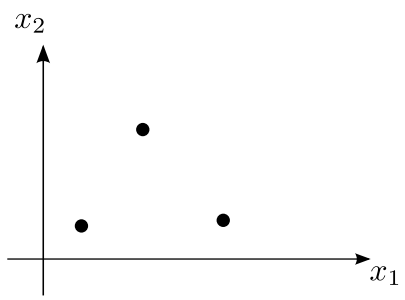
- Yes, as long as they're not colinear.

Q: *Can a perceptron classify any 4 points in 2D?*

- No, it cannot.

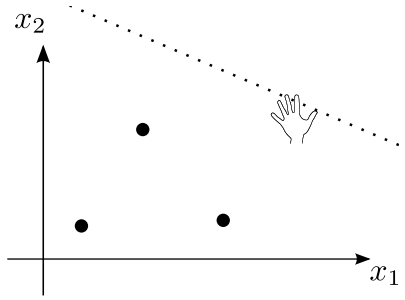
Shattering p points

1. Generate a dataset of p points in N dimensions and place them in some arrangement.



Shattering p points

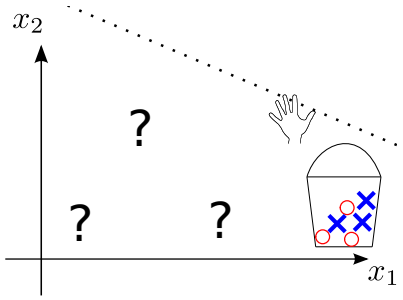
1. Generate a dataset of p points in N dimensions and place them in some arrangement.
2. Decide on a model class Λ (e.g. connectionist neuron with bias)



Shattering p points

1. Generate a dataset of p points in N dimensions and place them in some arrangement.
2. Decide on a model class Λ (e.g. connectionist neuron with bias)
3. For each possible binary label configuration (total: 2^p) $\{y_T^{(\alpha)}\}_{\alpha=1}^p$ do

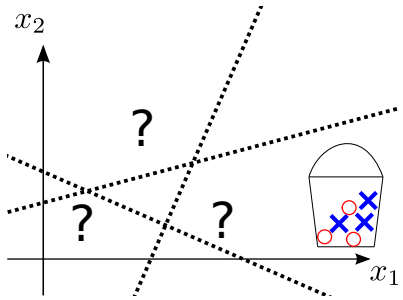
   Train the parameters $\underline{\mathbf{w}}$ of a model from the set Λ.
   Generate binary predictions $\{y^{(\alpha)}(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\}_{\alpha=1}^p$
   Measure the error $E^T_{[\underline{\mathbf{w}}]}$
   If $E^T_{[\underline{\mathbf{w}}]} = 0$ keep the model, otherwise, discard it.



Shattering p points

1. Generate a dataset of p points in N dimensions and place them in some arrangement.
2. Decide on a model class Λ (e.g. connectionist neuron with bias)
3. For each possible binary label configuration (total: 2^p) $\{y_T^{(\alpha)}\}_{\alpha=1}^p$ do

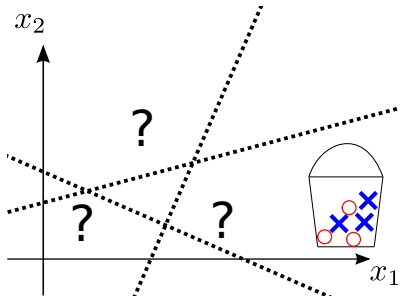
   Train the parameters $\underline{\mathbf{w}}$ of a model from the set Λ.
   Generate binary predictions $\{y^{(\alpha)}(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\}_{\alpha=1}^p$
   Measure the error $E^T_{[\underline{\mathbf{w}}]}$
   If $E^T_{[\underline{\mathbf{w}}]} = 0$ keep the model, otherwise, discard it.
4. Count how many models you've kept.



Shattering p points

1. Generate a dataset of p points in N dimensions and place them in some arrangement.
2. Decide on a model class Λ (e.g. connectionist neuron with bias)
3. For each possible binary label configuration (total: 2^p) $\{y_T^{(\alpha)}\}_{\alpha=1}^p$ do

   Train the parameters $\underline{\mathbf{w}}$ of a model from the set Λ.
   Generate binary predictions $\{y^{(\alpha)}(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\}_{\alpha=1}^p$
   Measure the error $E^T_{[\underline{\mathbf{w}}]}$
   If $E^T_{[\underline{\mathbf{w}}]} = 0$ keep the model, otherwise, discard it.



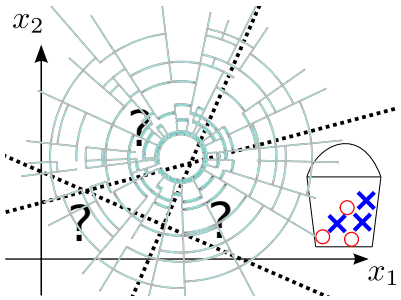
4. Count how many models you've kept.

If you were able to keep a model with $E^T_{[\underline{\mathbf{w}}]} = 0$ for each of the 2^p possible label configurations,
then the model class Λ with that number of tunable parameters in $\underline{\mathbf{w}}$ *shatters* a dataset of p points in N dimensions.

Shattering p points

1. Generate a dataset of p points in N dimensions and place them in some arrangement.
2. Decide on a model class Λ (e.g. connectionist neuron with bias)
3. For each possible binary label configuration (total: 2^p) $\{y_T^{(\alpha)}\}_{\alpha=1}^p$ do

   Train the parameters $\underline{\mathbf{w}}$ of a model from the set Λ.
   Generate binary predictions $\{y^{(\alpha)}(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\}_{\alpha=1}^p$
   Measure the error $E^T_{[\underline{\mathbf{w}}]}$
   If $E^T_{[\underline{\mathbf{w}}]} = 0$ keep the model, otherwise, discard it.



4. Count how many models you've kept.

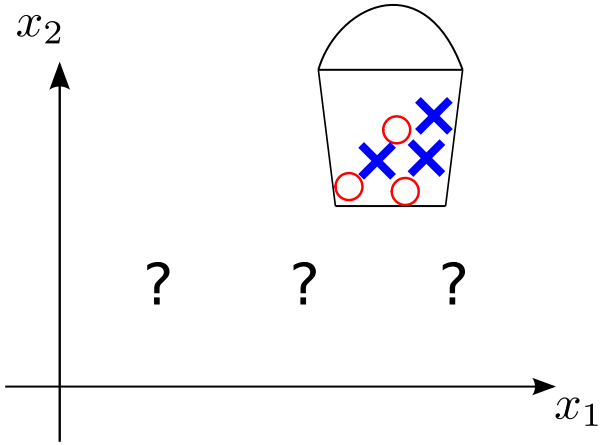
If you were able to keep a model with $E^T_{[\underline{\mathbf{w}}]} = 0$ for each of the 2^p possible label configurations,
then the model class Λ with that number of tunable parameters in $\underline{\mathbf{w}}$ *shatters* a dataset of p points in N dimensions.

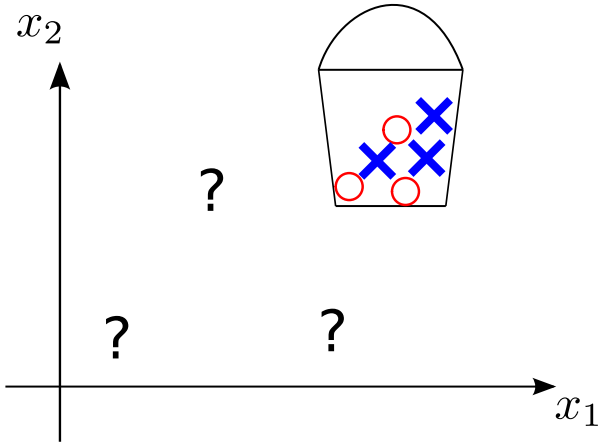
Shattering a dataset

Important:
Shattering requires finding at least one arrangement of the p points for which the model can be fitted to all 2^p label configurations.

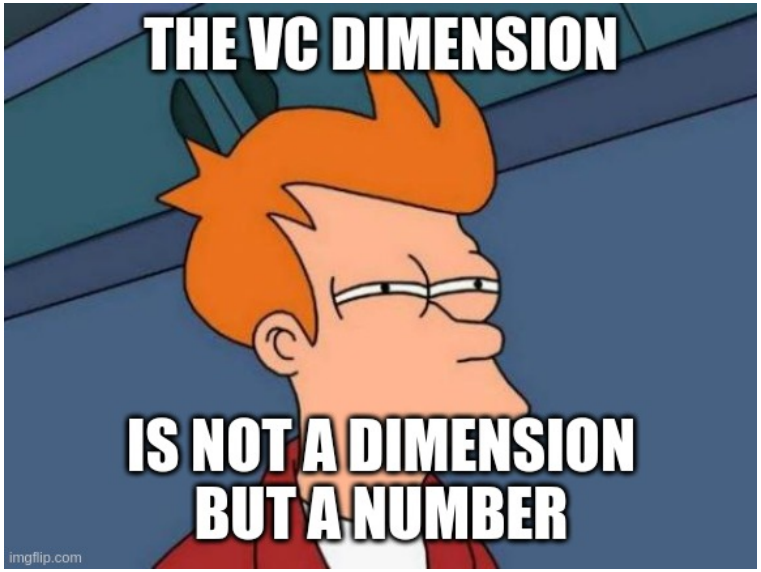
Shattering a dataset

Important:
Shattering requires finding at least one arrangement of the p points for which the model can be fitted to all 2^p label configurations.





The Vapnik-Chervonenkis (VC) dimension



$\mathrm{d_{VC}}$ is not actually a dimension but a single number *capacity measure* of a model class that is parameterized by $\underline{\mathbf{w}}$.

<u>Definition</u>:
$\mathrm{d_{VC}} :=$ <u>maximal number of data points p</u> that a model can *shatter*.

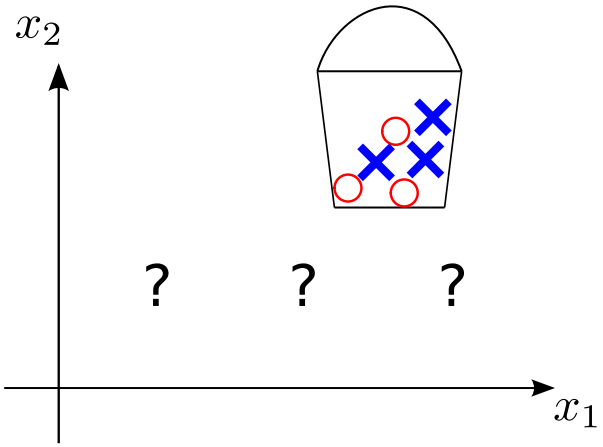
The Vapnik-Chervonenkis (VC) dimension

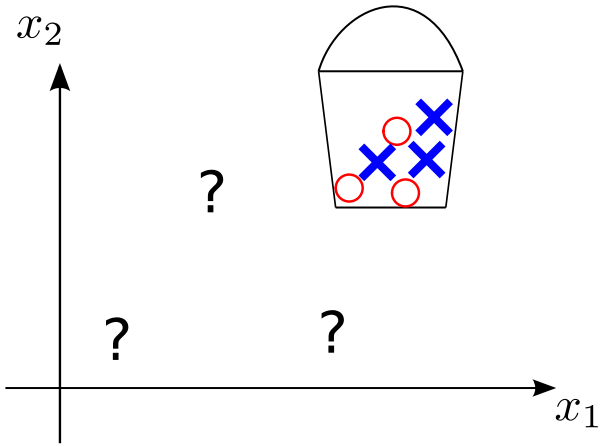
Definition:
In other words, The VC dimension is the largest # of points p for which there exist at least one dataset $\underline{\mathbf{X}} = \left(\underline{\mathbf{x}}^{(1)}, \ldots, \underline{\mathbf{x}}^{(p)}\right)^\top$ for which all 2^p label configurations can be solved with zero error.

The Vapnik-Chervonenkis (VC) dimension

Definition:

In more words: $\mathrm{d_{VC}}$ is the maximal number of data points p for which *all* possible label configurations $\{y_T^{(\alpha)} \in \{-1, 1\}\}_{\alpha=1}^{p}$ can be perfectly trained (i.e. $E^T_{[\underline{\mathbf{w}}]} = R_{\mathsf{emp}[\underline{\mathbf{w}}]} = 0$) by tuning the model parameters $\underline{\mathbf{w}}$. If (at least) one dataset $\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\}_{\alpha=1}^{p}$ with p points exists such that the model can be tuned to perfectly classify any label configuration for that dataset, then the $\mathrm{d_{VC}}$ for this model class is p.





Q: *What was the* $\mathrm{d_{VC}}$ *for our model in the previous example?*

Examples:

$$
\begin{aligned}
{\mathrm{d_{VC}}}^{\text{perceptron}} &= N + 1 = \#\text{params} \\
{\mathrm{d_{VC}}}^{\text{MLP}} &= \mathcal{O}(\#\text{params}^2) \\
{\mathrm{d_{VC}}}^{\text{1NN}} &= \infty \\
{\mathrm{d_{VC}}}^{\text{sinusoid}} &= \infty
\end{aligned}
$$

$$
y^{\text{sinusoid}}(x; w) = \mathrm{sign}\left[\sin\left(wx\right)\right]
$$

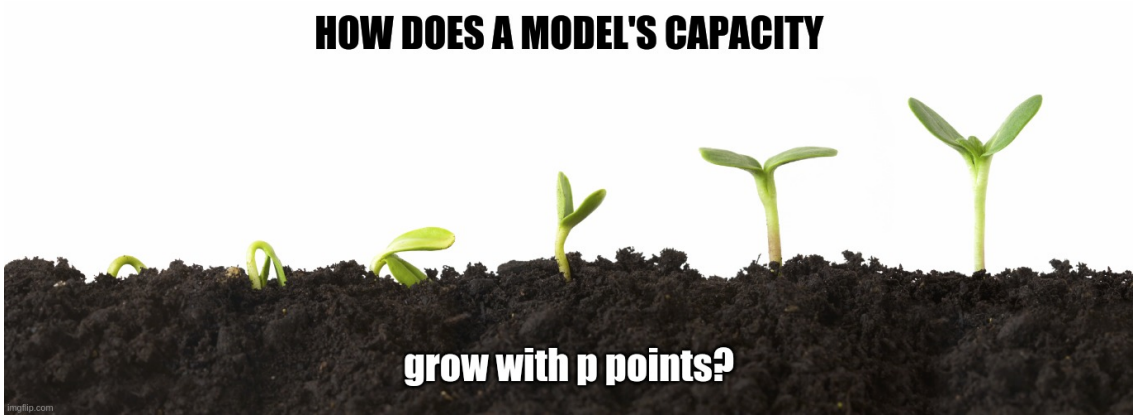
Examples:

$$
\begin{aligned}
{\mathrm{d_{VC}}}^{\text{perceptron}} &= N + 1 = \#\text{params} \\
{\mathrm{d_{VC}}}^{\text{MLP}} &= \mathcal{O}(\#\text{params}^2) \\
{\mathrm{d_{VC}}}^{\text{1NN}} &= \infty \\
{\mathrm{d_{VC}}}^{\text{sinusoid}} &= \infty
\end{aligned}
$$

$$
y^{\text{sinusoid}}(x; w) = \mathrm{sign}\left[\sin\left(wx\right)\right]
$$

Q: *How is* ${\mathrm{d_{VC}}}^{\textit{sinusoid}} = \infty$*? There's only 1 parameter!*

The growth function



The growth function - What is it?

The growth function is the measure of *sample* complexity for a model class.

It measures the complexity of the functions that can be fitted using a model from that model class (e.g. connectionist neuron).

sample complexity: *how much can I do with p points in N dimensions?*

It is closely related to the *VC dimension*.

The growth function $G^{\Lambda}_{(p)}$

- **data representation:** $\underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \{-1, +1\}$
- **model class:** set Λ of functions $y_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} \in \{-1, +1\}$
- **binary label vector:** $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})} = \left(y_{(\underline{\mathbf{x}}^{(1)},\underline{\mathbf{w}})}, y_{(\underline{\mathbf{x}}^{(2)},\underline{\mathbf{w}})}, \ldots, y_{(\underline{\mathbf{x}}^{(p)},\underline{\mathbf{w}})}\right)$
  - different classifiers can induce the same label vector on the training set
- **number of different vectors** $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})}$ induced by all $\underline{\mathbf{w}} \in \Lambda$:

$$N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})} \quad \leq 2^p$$

The growth function $G^{\Lambda}_{(p)}$

- **data representation:** $\underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \{-1, +1\}$

- **model class:** set Λ of functions $y_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} \in \{-1, +1\}$
  Think of Λ as a set of classifiers. The classifiers in this set come from the same model class (e.g. connectionist neurons with weights $\underline{\mathbf{w}}$). Each classifier describes a different hyperplane (different $\underline{\mathbf{w}}$) to separate the two classes in the data.

The growth function $G^{\Lambda}_{(p)}$

- **data representation:** $\underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \{-1, +1\}$

- **model class:** set Λ of functions $y_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} \in \{-1, +1\}$
  Think of Λ as a set of classifiers. The classifiers in this set come from the same model class (e.g. connectionist neurons with weights $\underline{\mathbf{w}}$). Each classifier describes a different hyperplane (different $\underline{\mathbf{w}}$) to separate the two classes in the data.

- **binary label vector**: $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})} = \left(y_{(\underline{\mathbf{x}}^{(1)},\underline{\mathbf{w}})}, y_{(\underline{\mathbf{x}}^{(2)},\underline{\mathbf{w}})}, \ldots, y_{(\underline{\mathbf{x}}^{(p)},\underline{\mathbf{w}})}\right)$
  - different classifiers can induce the same label vector on the training set

  Each classifier in Λ describes a different hyperplane.

The growth function $G^{\Lambda}_{(p)}$

- **data representation:** $\underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \{-1, +1\}$

- **model class:** set Λ of functions $y_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} \in \{-1, +1\}$
  Think of Λ as a set of classifiers. The classifiers in this set come from the same model class (e.g. connectionist neurons with weights $\underline{\mathbf{w}}$). Each classifier describes a different hyperplane (different $\underline{\mathbf{w}}$) to separate the two classes in the data.

- **binary label vector**: $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})} = \left(y_{(\underline{\mathbf{x}}^{(1)},\underline{\mathbf{w}})}, y_{(\underline{\mathbf{x}}^{(2)},\underline{\mathbf{w}})}, \ldots, y_{(\underline{\mathbf{x}}^{(p)},\underline{\mathbf{w}})}\right)$
  - different classifiers can induce the same label vector on the training set

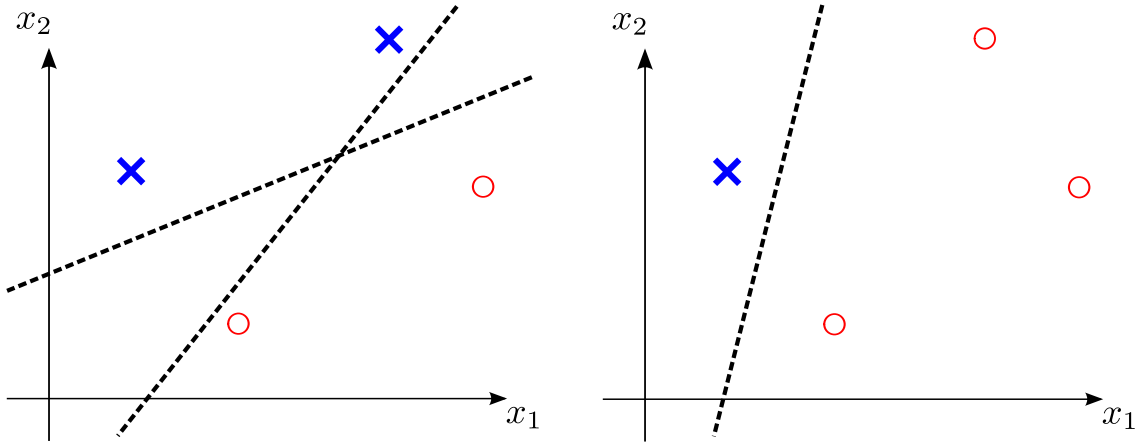
  Each classifier in Λ describes a different hyperplane.

- **number** of different vectors $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})}$ induced by all $\underline{\mathbf{w}} \in \Lambda$:

$$N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})} \leq 2^p$$

The growth function $G^{\Lambda}_{(p)}$ and N^{Λ}

Two hyperplanes can be different but still yield the same predictions.
Example:



$$N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})} \leq 2^p$$

Defining the growth function $G^{\Lambda}_{(p)}$

$$G^{\Lambda}_{(p)} = \ln\left(\sup_{\underline{\mathbf{x}}^{(1)},\ldots\underline{\mathbf{x}}^{(p)}} N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})}\right) = \ln 2^p = p\ln 2$$

Recall:

$$\lim_{p\to\infty} P\left\{\left|R_{[\underline{\mathbf{w}}_p]} - R_{[\underline{\mathbf{w}}_0]}\right| \geq \eta\right\} \;=\; 0\,, \quad \forall\eta > 0$$

This convergence is equivalent to[2]:

$$\lim_{p\to\infty} G^{\Lambda}_{(p)}/p \stackrel{!}{=} 0$$

$$\lim_{p\to\infty} (p\ln 2)\,/p = \ln 2 \neq 0$$

the classifier's predictions become ambiguous and will likely fail to generalize.

[2] If interested, see Ch. 5.5 (Scholkopf and Smola, 2001) for a description of the approach and (Vapnik, 1999) for the detailed derivation.

Complete ambiguity

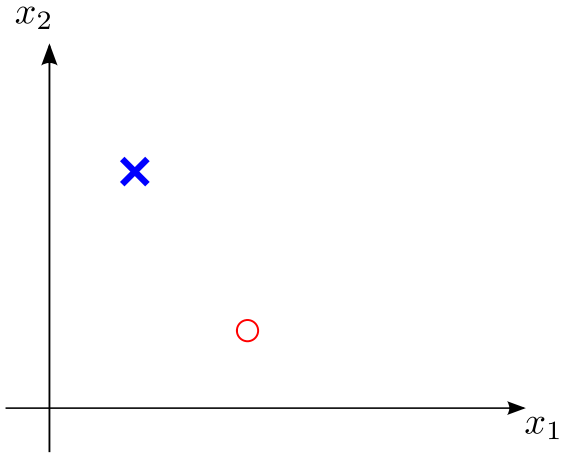
The growth function does not tell us how many points we need to separate the two classes but how many points we need to exclude completely ambiguous predictions.

Example with complete ambiguity

Linear neuron, 2 points in 2D. We do not have access to the hyperplane $\underline{\mathbf{w}}$.
We only have access to the predictions made by the neuron:

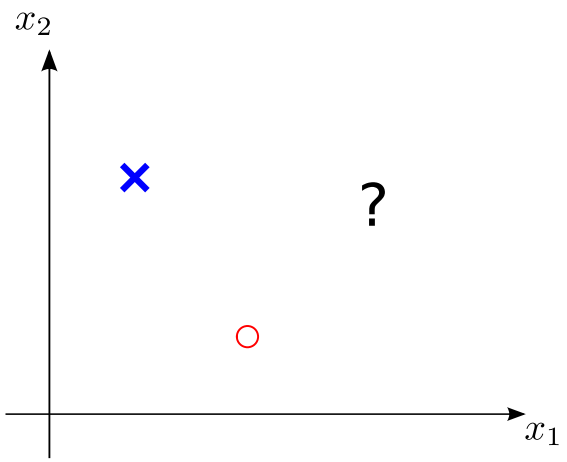
Example with complete ambiguity

Linear neuron, 2 points in 2D. We do not have access to the hyperplane $\underline{\mathbf{w}}$. We only have access to the predictions made by the neuron:



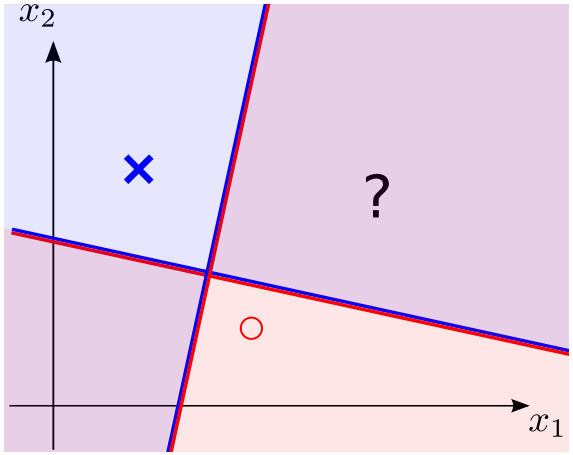
Example with complete ambiguity

Linear neuron, 2 points in 2D. We do not have access to the hyperplane $\underline{\mathbf{w}}$. We only have access to the predictions made by the neuron:



Example with complete ambiguity

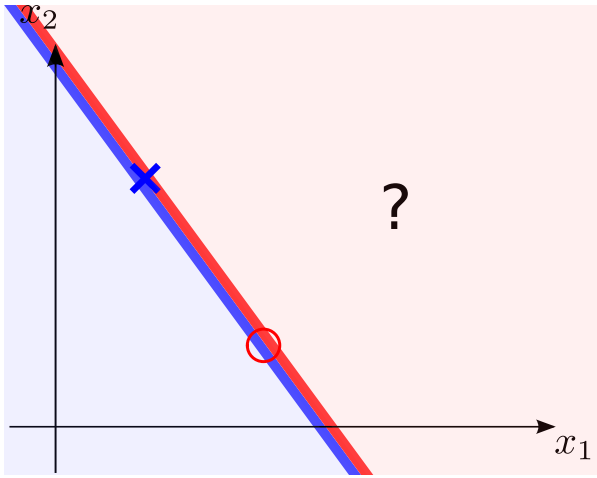
Linear neuron, 2 points in 2D. We do not have access to the hyperplane $\underline{\mathbf{w}}$. We only have access to the predictions made by the neuron:

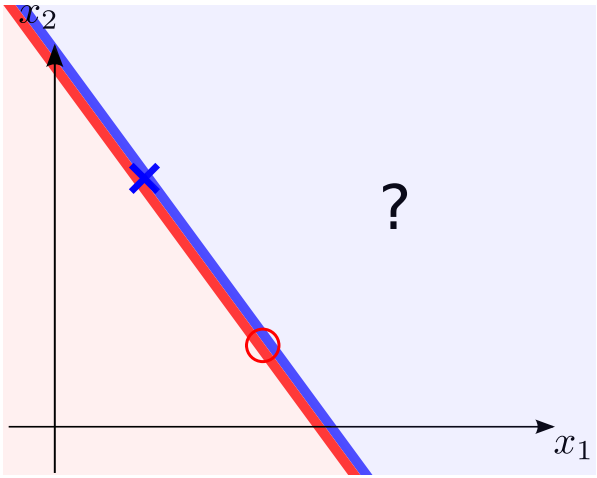


Ambiguity not **yet** complete.

Example with complete ambiguity

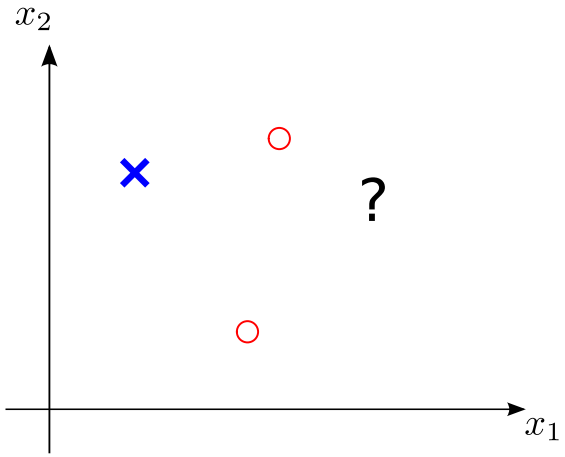
Linear neuron, 2 points in 2D. We do not have access to the hyperplane $\underline{\mathbf{w}}$. We only have access to the predictions made by the neuron:





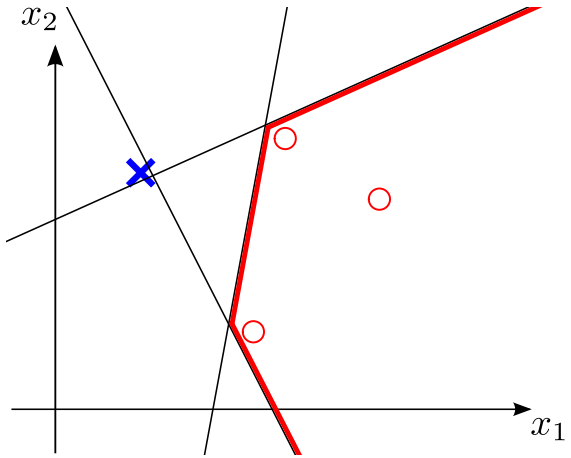
Example with **non**complete ambiguity

Linear neuron, 3 points in 2D:



Example with **non**complete ambiguity

Linear neuron, 3 points in 2D:

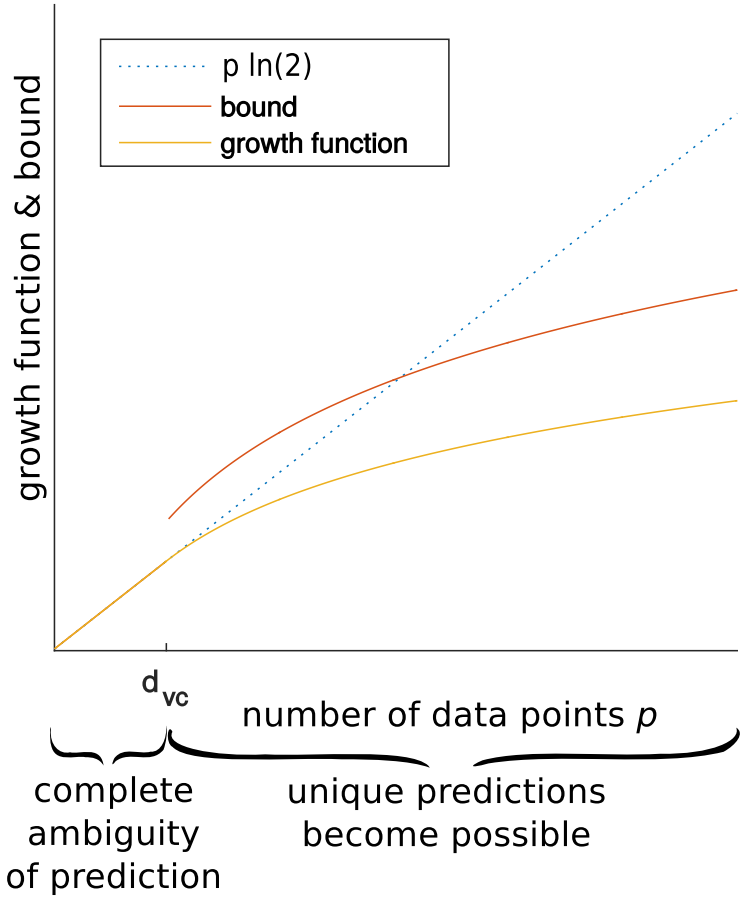


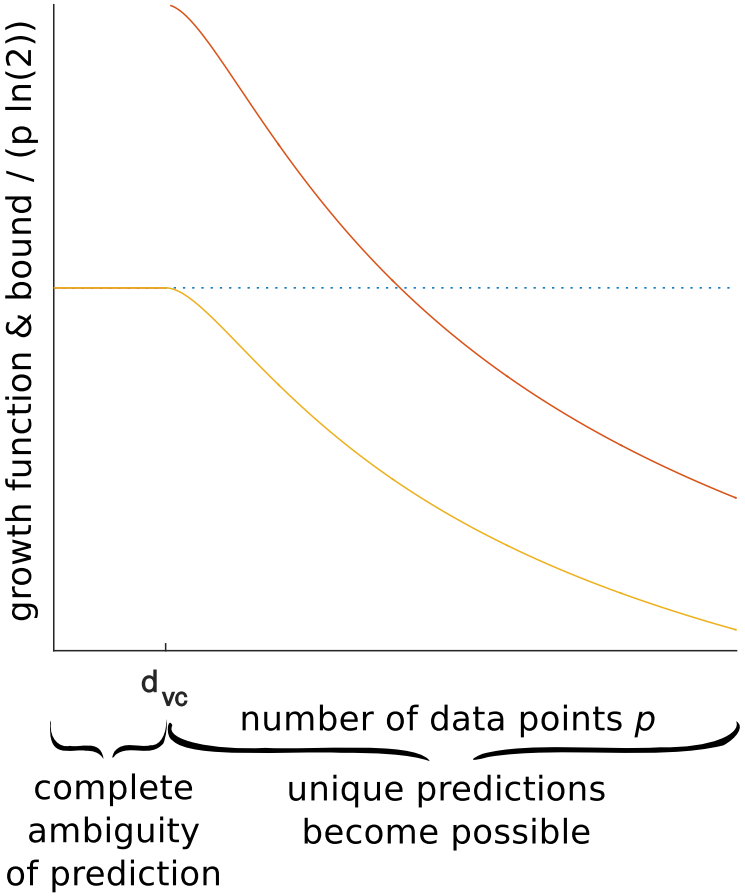
- bound on growth function Vapnik (1998)

$$G^{\Lambda}_{(p)} \begin{cases} = p \ln 2 & \text{for } p \leq \mathrm{d_{VC}} \\ \leq \mathrm{d_{VC}} \cdot \left(1 + \ln \frac{p}{\mathrm{d_{VC}}}\right) & \text{for } p > \mathrm{d_{VC}} \end{cases}$$

- Vapnik-Chernovenkis dimension $\mathrm{d_{VC}}$:
  capacity measure of the model class

The growth function $G^{\Lambda}_{(p)}$



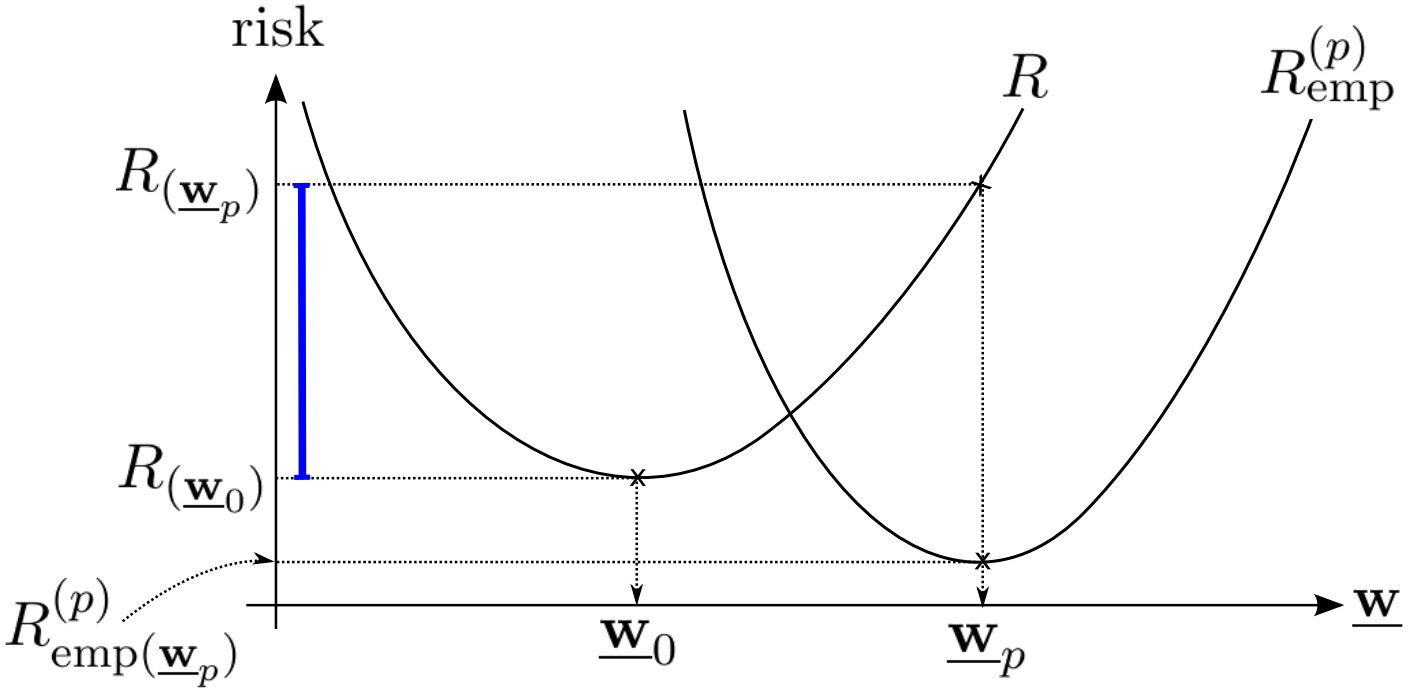


$\mathrm{d_{VC}}$: capacity measure of the model class

small $\mathrm{d_{VC}}$: small sample size sufficient for learning

large $\mathrm{d_{VC}}$: large sample size required

The solution to learning problem 2

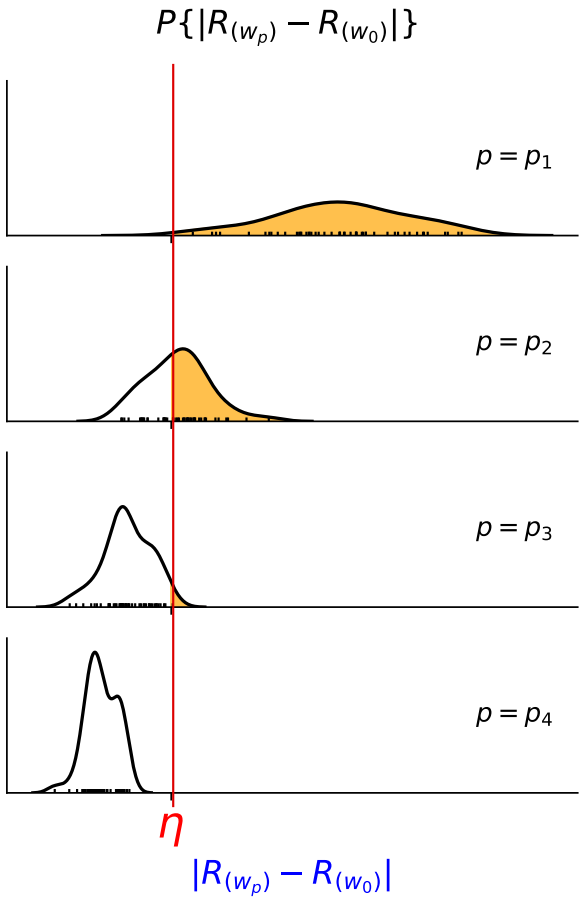


2. **finite samples:** deviation from the optimal model

$$P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| > \left(\frac{G^{\Lambda}_{(2p)} - \ln\frac{\epsilon}{8}}{p}\right)^{\frac{1}{2}} + \left(-\frac{\ln\frac{\epsilon}{2}}{2p}\right)^{\frac{1}{2}} + \frac{1}{p}\right\} < \epsilon$$

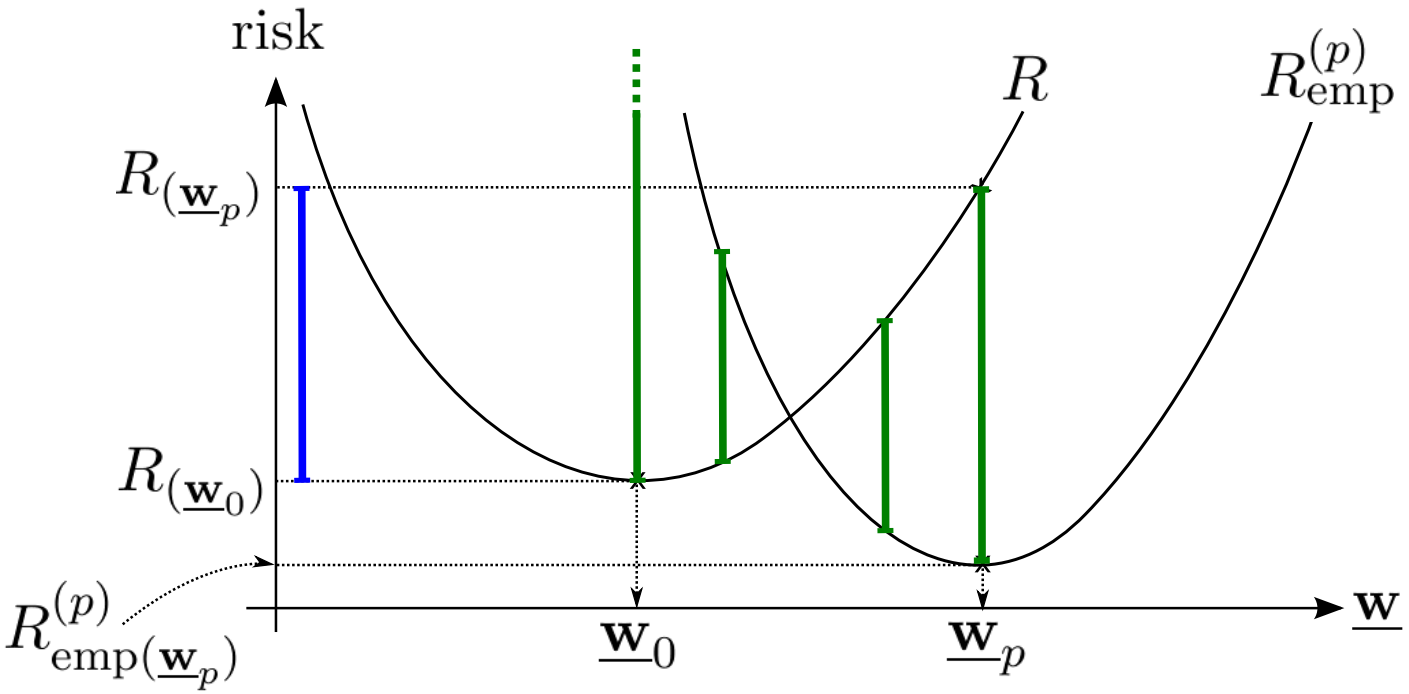
- finite $\mathrm{d_{VC}}$: increased sample size $p \Rightarrow$ reduction of the bound

Re-thinking η: bounded by the growth function



$$P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| > \left(\frac{G^{\Lambda}_{(2p)} - \ln\frac{\epsilon}{8}}{p}\right)^{\frac{1}{2}} + \left(-\frac{\ln\frac{\epsilon}{2}}{2p}\right)^{\frac{1}{2}} + \frac{1}{p}\right\} < \epsilon$$

The solution to learning problem 3



3. **finite samples:** bound on the generalization error

$$P\left\{\sup_{\underline{\mathbf{w}}\in\Lambda}\left|R_{(\underline{\mathbf{w}})}-R^{(p)}_{\mathrm{emp}(\underline{\mathbf{w}})}\right|>\eta\right\}<4\exp\left(G^{\Lambda}_{(2p)}-p\left(\eta-\tfrac{1}{p}\right)^2\right)$$

- bound non-trivial only if $G^{\Lambda}_{(2p)}$ is sub-linear in p

Results of Statistical Learning Theory

- formulation of conditions under which ERM works and its convergence
- bounds describing **generalization ability** of ERM
  For a finite training set, the generalization error is bound with probability $1-\epsilon$:

$$E^G_{[\underline{\mathbf{w}}]} \;\leq\; E^T_{[\underline{\mathbf{w}}]} + \sqrt{\frac{\mathrm{d_{VC}}\left(\ln\frac{2\,p}{\mathrm{d_{VC}}}+1\right)-\ln\frac{\epsilon}{4}}{p}} + \frac{1}{p} \quad \text{for } p > \mathrm{d_{VC}}$$

  100% overfitting for $p < \mathrm{d_{VC}}$.
  Bound allows calculating p to target a given generalization error.
- inductive inference for **small sample sizes** based on these bounds
- methods for implementing this new type of inference ($\rightarrow$ maximum margin classifiers e.g. SVMs)

References I

Bernhard Scholkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.

Vladimir Vapnik. *The Support Vector Method of Function Estimation*, pages 55–85. Springer US, Boston, MA, 1998. ISBN 978-1-4615-5703-6. doi: 10.1007/978-1-4615-5703-6_3. URL https://doi.org/10.1007/978-1-4615-5703-6_3.

Vladimir N Vapnik. An overview of statistical learning theory. *IEEE transactions on neural networks*, 10(5):988–999, 1999.